JANMITRA

Civic Issue Reporting & Management System

Nagpur Municipal Corporation | 4 Categories | 4 Wards | 4 Departments

Team ResolveIT

System Architecture Document

How all components connect and interact

CITIZEN PORTAL	Report Issue | Track Status | Verify Resolution
BACKEND (Django)	REST API | Auth | Business Logic | AI Module
AI VALIDATION	Fake Detection (EXIF+ELA+HuggingFace) | YOLO Relevance Check
DATABASE	PostgreSQL + PostGIS | 4 Categories | 4 Wards | 4 Depts
DEPT. DASHBOARD	PWD | Solid Waste | Water Works | Electrical

Anas Shaikh | Anurag Deshmukh | Om Gadekar | Faizan Rayeen

September 2026 | B.Tech Final Year Project

1. Architecture Overview

JANMITRA is a three-tier web application. The React frontend talks to the Django backend via REST API. Django talks to PostgreSQL for all data. Heavy work (AI validation, notifications) runs asynchronously via Celery + Redis so users never wait. Four Django apps handle distinct concerns: accounts, complaints, dashboard, notifications.

```
BROWSER (React)          BACKEND (Django)           DATABASE (PostgreSQL)
    |                         |                          |
Citizen Portal  --HTTP-->  DRF API Views  --ORM-->  complaints table
Worker Panel    --HTTP-->  Auth (JWT)     --ORM-->  users table
Officer Panel   --HTTP-->  Business Logic --ORM-->  categories/wards
Admin Panel     --HTTP-->  Celery Dispatch           departments table
    |                         |                          |
Leaflet Maps    <--JSON--  Dashboard APIs --ORM-->  analytics queries
Recharts        <--JSON--  Celery Worker  --ORM-->  notifications table
Firebase JS     <------- Firebase Admin --FCM--> Browser Push

Celery Worker reads from Redis queue, runs AI pipeline:
  [EXIF check] -> [ELA check] -> [HuggingFace] -> [YOLO] -> [Duplicate] -> [Route]
```

2. Django App Structure

accounts	Custom User model (mobile login, role, ward, department, designation). Registration, login, JWT, RBAC permission classes: IsCitizen, IsWorker, IsDeptOfficer, IsAdmin.
complaints	Category model (4 only), Ward model (4 only), Complaint model, ComplaintImage, StatusHistory. All complaint APIs. Worker task API. Citizen verification API. AI Celery task chain. Assignment API with dept+ward validation.
dashboard	KPI API, Ward-stats API, Trend API, Worker workload API, High-priority list API, Pending validation queue. All filtered by department for officers, all-access for admin.
notifications	Notification model, Firebase Admin SDK, send_push_notification Celery task wired to all status changes.

3. Department + Ward Architecture

INFO	The department and ward relationship is the core of JANMITRA's routing logic. Every entity in the system belongs to a department and/or ward. This architecture ensures complaints always go to the right person.

```
CATEGORY (4 total, static)
  Roads & Infrastructure    -> department_id = 1 (PWD)
  Garbage & Waste Mgmt      -> department_id = 2 (SWM)
  Water & Drainage          -> department_id = 3 (PHE/Water Works)
  Street Lighting/Elec      -> department_id = 4 (Electrical)

WARD (4 total, static)
  Ward 1, Ward 2, Ward 3, Ward 4

COMPLAINT ROUTING FLOW:
  1. Citizen selects category -> department auto-set
  2. Citizen selects ward -> ward stored on complaint
  3. Celery validates -> routes to department queue
  4. Officer assigns -> ONLY workers where:
       worker.department_id == complaint.department_id
       worker.ward_id       == complaint.ward_id
  5. Admin can override both checks (ward_override=True)
```

4. AI Module Architecture

4.1 Two-Layer Validation Pipeline

Layer	Step	Tool	Time	Action on Failure
1	EXIF Metadata	Pillow	0.1s	Reject: Fake Image (no camera data)
1	ELA Analysis	OpenCV+NumPy	0.5s	Reject: Fake Image (too uniform)
1	AI Fake Detector	Transformers	3-5s	Reject: AI-generated image detected
2	YOLO v8	Ultralytics	5-10s	Pending Validation (human review)
-	Duplicate Check	PostGIS	0.2s	Mark as duplicate, link to parent
-	Priority Score	Python formula	0.1s	Score and level saved
-	Auto-Route	Django ORM	0.1s	Department assigned, citizen notified

4.2 Priority Scoring Formula

Score = (Severity x 0.30) + (Safety x 0.25) + (Public Impact x 0.20) + (Duplicate Count x 10 x 0.15) + (Dept Capacity x 0.10)

Priority Level	Score Range	Action
Critical	81-100	Immediate. Alert dept. officer. Top of worker task list.
High	61-80	Fix within 24-48 hours.
Medium	31-60	Fix within this week.
Low	0-30	Fix when capacity allows.

5. Deployment Architecture

5.1 Local Development (Docker Compose)

WARN All 4 team members MUST use Docker. Celery does not work on Windows natively. Docker provides identical Linux environment for all.

Service	Docker Image	Port
postgres (PostGIS)	postgis/postgis:15-3.3	5432
redis	redis:7-alpine	6379
backend (Django)	Python 3.11 custom	8000
celery worker	Same as backend	--
celery beat	Same as backend	--
frontend (React)	Node 18 custom	5173

5.2 Production (Render Free Tier)

Render Service	Type	Runs
resolveit-backend	Web Service	Django + Gunicorn
resolveit-celery	Background Wkr	Celery AI + notification worker
resolveit-frontend	Static Site	Built React (npm run build -> dist/)
resolveit-db	Render DB	PostgreSQL 15 + PostGIS
Redis Cloud	External Free	Redis broker (redislabs.com, 30MB)

NOTE UptimeRobot (free): pings backend URL every 10 minutes to prevent Render Free Tier sleep during demo.

6. Security Architecture

Layer	Mechanism
Authentication	JWT: access token 60 min, refresh token 7 days (SimpleJWT)
Authorisation	RBAC: IsCitizen, IsWorker, IsDeptOfficer, IsAdmin on every endpoint
Worker Assignment	API validates dept+ward match before assignment. DB constraint as backup.
Fake Image Block	AI rejection before complaint enters the system. No DB record created.
Data Isolation	Dept. Officer API always filters by officer's department_id. No cross-dept data.
Worker Isolation	Worker task API always filters by worker's dept_id AND ward_id.
HTTPS	Enforced by Render in production. No plain HTTP.
Secrets	All keys (DB URL, Redis URL, FCM, SECRET_KEY) in env variables. Never in code.